macOS Container Manager (containerd / Apple container) - Software Specification & Build Guide

1. Overview

This document defines the architecture, features, and build process for a macOS application designed to manage containers using Apple's container runtime. The application aims to replicate and improve upon Docker Desktop functionality using a native Apple stack.

2. Goals

1. Provide a native macOS GUI for container management
2. Support container lifecycle operations (start, stop, logs, exec)
3. Enable image management (pull, build, delete)
4. Provide system monitoring and resource visibility
5. Offer future extensibility for multi-container orchestration

3. Architecture

The application is composed of a SwiftUI frontend, a service layer for container operations, and a CLI/API integration layer interfacing with Apple's container runtime.

1. Frontend: SwiftUI-based macOS application
2. Service Layer: Business logic and state management
3. CLI Layer: Wrapper around `container` command
4. Runtime: Apple Containerization Framework + Virtualization

4. Core Features

Container Management

1. List containers
2. Start/Stop/Restart containers
3. View logs (real-time)
4. Execute commands inside container
5. View resource usage

Image Management

1. Pull images from registry
2. Build images from Containerfile
3. Delete and tag images

System Management

1. Start/Stop container system
2. View disk usage
3. View network configuration

Networking

1. Port mapping UI
2. Basic network visualization

Volumes

1. Manage bind mounts
2. Persistent storage UI

5. Technology Stack

1. Frontend: SwiftUI
2. Language: Swift
3. Runtime: Apple container / containerd
4. Optional: Combine / Observation framework
5. Storage: Local state + Keychain for credentials

6. Project Structure

mac-container-manager/ - App/ - Core/ - CLI/ - Models/ - ViewModels/ - Utils/

7. Build Guide

Step 1: Install Dependencies

1. Install Xcode (latest)
2. Install Apple container CLI
3. Verify container system: container system start

Step 2: Create Project

1. Open Xcode
2. Create new macOS SwiftUI app
3. Set bundle identifier

Step 3: CLI Integration

1. Create a Swift class to execute shell commands
2. Wrap `container` CLI commands
3. Parse outputs (JSON or text)

Step 4: Build UI

1. Sidebar navigation
2. Container list view
3. Detail view with logs and metrics

Step 5: Implement Features

1. Container list and actions
2. Image management
3. Logs streaming
4. Terminal execution

Step 6: Testing

1. Test container lifecycle operations
2. Test UI responsiveness
3. Test error handling

Step 7: Packaging

1. Build release version
2. Sign app with Apple Developer account
3. Distribute via DMG or TestFlight

8. Future Enhancements

1. Compose-like orchestration
2. AI-powered container setup
3. Advanced networking visualization
4. Kubernetes integration (optional)